

EE-170L: Computer Programming Lab

Laboratory Report — Session 09

Introduction to Functions in C++

Lab No.:	09	Date:	May 11, 2026
Course:	EE-170L Computer Programming Lab	Semester:	Spring 2026
Institute:	UET Peshawar	Department:	Electrical Engineering

1. Objective

The objective of this lab session is to understand and implement user-defined functions in C++. Students will learn the concept of function declaration (prototype), function definition, and function calling. The lab also covers passing parameters to functions and returning values from functions.

2. Theory

2.1 What is a Function?

A function is a named block of code that performs a specific task. Functions allow programmers to divide a long program into manageable, reusable sections. In C++, every function has a name (identifier), may accept parameters (arguments), and may return a value. The most commonly encountered function is **main()**, which serves as the entry point of every C++ program.

2.2 Three Parts of a Function

Part	Description	Example
Prototype (Declaration)	Tells the compiler the function exists before it is used. Must appear before use.	<code>int firstAsterisk();</code>
Function Call	Executes the function from inside <code>main()</code> or another function.	<code>firstAsterisk();</code>
Function Definition	Contains the actual code/body that runs when the function is called.	<code>int firstAsterisk() { ... }</code>

2.3 Function Prototype Syntax

The prototype of a function specifies the return type, function name, and parameter types so the compiler can verify correct usage before the function definition is seen.

```
returnType functionName(parameterType1, parameterType2, ...);
```

```
// Examples:
```

```
int main(); // No parameters, returns int
```

```
float divide(int a, int b); // Two int parameters, returns float
```

```
void square(float x); // One float parameter, returns nothing
```

2.4 Return Types

Return Type	Meaning	Usage
void	Function returns nothing	<code>void greet() { ... }</code>
int	Function returns an integer	<code>int add(int a, int b) { return a+b; }</code>
float	Function returns a decimal	<code>float avg(int s, int n) { return (float)s/n; }</code>

3. Lab Examples (from Lab Manual)

Example 1: Function with No Parameters (lab10p01.cpp)

This program demonstrates a basic void function with no parameters. The function **asterisk()** prints a row of 10 asterisks when called from main().

```
#include <iostream>
using namespace std;

void asterisk(); // Function prototype

int main() {
    asterisk(); // Function call
    return 0;
}

// Function definition
void asterisk() {
    for (int i = 0; i < 10; i++) {
        cout << "*";
    }
}
```

Output:

```
*****
```

Example 2: Function with One Parameter (lab10p02.cpp)

This example shows a function that accepts a single integer argument and prints that many asterisks, demonstrating how parameters pass data into a function.

```
#include <iostream>
using namespace std;

void asterisk(int n); // Prototype: takes one integer

int main() {
    asterisk(12); // Call with argument 12
    return 0;
}

void asterisk(int num) {
    for (int i = 0; i < num; i++) {
```

```
cout << "*";  
}  
}
```

Output:

```
*****
```

Example 3: Function with Two Parameters & Return Value (lab10p03.cpp)

This example demonstrates a function that accepts two integer parameters, computes their sum, and returns the result to the caller using the **return** statement.

```
#include <iostream>  
using namespace std;  
  
int add(int, int); // Prototype: takes two ints, returns int  
  
int main() {  
    int result;  
    result = add(3, 5); // Call; result receives the returned value  
    cout << result;  
    return 0;  
}  
  
int add(int num1, int num2) {  
    return num1 + num2; // Return the sum  
}
```

Output:

```
8
```

4. Lab Tasks

Task 1: Function Introduction — greet()

Problem Statement: Declare a function called **greet** that takes no parameters and returns nothing. Inside the function, display a greeting message of your choice. Call the function from `main()`.

```
#include <iostream>  
using namespace std;  
  
void greet(); // Prototype  
  
int main() {  
    greet(); // Function call  
    return 0;  
}  
  
// Function definition  
void greet() {  
    cout << "Welcome to EE-170L Computer Programming Lab!" << endl;  
    cout << "Have a great lab session today." << endl;  
}
```

```
}
```

Output:

```
Welcome to EE-170L Computer Programming Lab!
```

```
Have a great lab session today.
```

Task 2: Basic Function — sayHello()

Problem Statement: Declare a function called **sayHello** that takes no parameters and returns nothing. Display the message "Hello, World!" inside it and call it from main().

```
#include <iostream>
using namespace std;

void sayHello(); // Prototype

int main() {
    sayHello(); // Function call
    return 0;
}

void sayHello() {
    cout << "Hello, World!" << endl;
}
```

Output:

```
Hello, World!
```

Task 3: Array Sum using calculateSum() Function

Problem Statement: Write a C++ program that takes 10 integer values from the user and stores them in a 1-D array. Create a function **calculateSum** that accepts the array and its size as parameters and returns the sum of all elements. In main(), call the function and also calculate and display the average.

Constraints: Use only 1-D array. Use a function for sum calculation.

```
#include <iostream>
using namespace std;

// Prototype: accepts array and its size, returns the sum
int calculateSum(int arr[], int size);

int main() {
    int arr[10];

    // Take 10 integers from the user
    cout << "Enter 10 integer values:" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Value " << i + 1 << ": ";
        cin >> arr[i];
    }

    // Call function to get the sum
    int sum = calculateSum(arr, 10);
```

```

// Calculate average
float average = (float)sum / 10;

// Display results
cout << "\nSum of all elements : " << sum << endl;
cout << "Average of all elements : " << average << endl;

return 0;
}

// Function definition
int calculateSum(int arr[], int size) {
int sum = 0;
for (int i = 0; i < size; i++) {
sum += arr[i]; // Accumulate each element
}
return sum;
}

```

Sample Input / Output:

Enter 10 integer values:

Value 1 : 5

Value 2 : 10

Value 3 : 15

Value 4 : 20

Value 5 : 25

Value 6 : 30

Value 7 : 35

Value 8 : 40

Value 9 : 45

Value 10: 50

Sum of all elements : 275

Average of all elements : 27.5

Explanation: The function `calculateSum()` receives the array by reference (arrays are always passed by address in C++) along with its size. It iterates through each element using a for-loop, accumulates the total in a local variable `sum`, and returns it. The average is computed in `main()` by casting the integer `sum` to float before dividing.

5. Results & Observations

Task	Function Used	Parameters	Return Type	Output / Result
1	<code>greet()</code>	None	void	Greeting message printed
2	<code>sayHello()</code>	None	void	Hello, World! printed
3	<code>calculateSum()</code>	<code>int arr[], int size</code>	int	Sum = 275, Avg = 27.5

6. Conclusion

In this lab session, we successfully learned and applied the concept of user-defined functions in C++. The following key concepts were covered and practised:

- Functions modularize code, making programs easier to read, debug, and reuse.
- Every function in C++ must have a prototype (declaration) before it is called.
- Functions can have zero or more parameters and may return a value or void.
- The **return** statement sends a value back to the calling function.
- Arrays passed to functions are handled by address, so no explicit pointer syntax is needed for 1-D arrays.
- Separating logic (sum calculation) from output (main) is good programming practice.

Student Signature:

Lab Instructor Signature:

Marks Obtained: _____ / 10